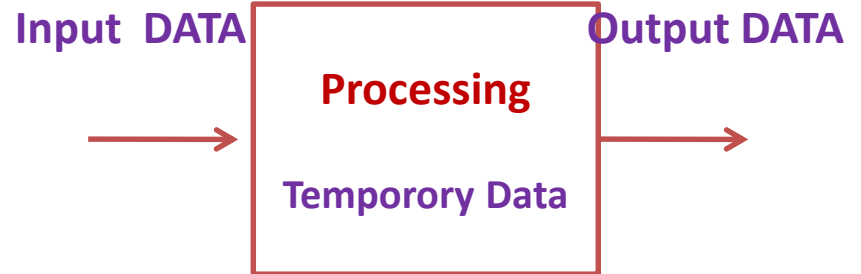


Basics of Algorithmic

L.Djaafri

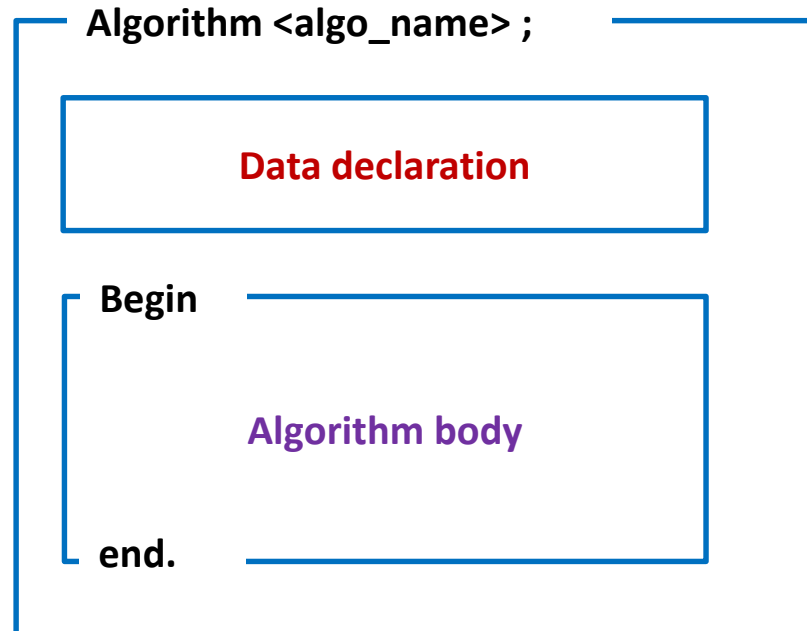
Algorithm structure

An algorithm is used to **process** the problem's **input data** to arrive at the required results: **output data**. During processing, **temporary data** may be required



The **declaration section** identifies all the **data** used in the algorithm

The **body part** is dedicated to **processing** performed by a **sequence of elementary instructions**



Data declaration

Two types of data are used in an algorithm:

→ **Constants**

→ **Variables**

Notion d'identificateur

An identifier is a name chosen by the programmer to designate constants, variables and other computer objects.

An identifier is made up of a sequence of characters respecting a set of rules:

generally uses alphanumeric and symbolic characters Does not begin with a digit

Does not contain the characters :+, -, *, / ,. , ?

Do not use keywords already in use, such as: if, for, end, ...

Exemple:

<code>x, vitesse_vent, val, nomPersonne</code>	Correct identifier
<code>12x, vitesse vent, nom-Personne</code>	Incorrect identifier

Constants

A constant is data that does not change value during execution of the algorithm.

A constant can never be modified

its value must be specified when the object is defined.

Syntax:

CONST

`<identifiant>=<valeur>;`

Example:

CONST

`PI = 3.14;`

`NOM = « Pascal »;`

Variables

A variable is a data with a name that can change value during program execution.

A variable is characterized by :

- Name (identifier)
- Data type
- Value that can change
- Memory area whose size is defined by variable type

Syntax:

Var

```
<identifiant>:<type1>;  
<identifiant1>,<identifiant2>:<type2>;
```

Example:

Var

```
a, temp      :integer;  
surf         :real;
```

Data types

The type of a variable defines:

- The set of values the variable or constant can take.
- The set of operations that can be applied to the variable.

These basic types are often:

- **Boolean type:** Values of true or false.
- **Integer type** (signed or unsigned): Values encoded in 8 bits, 16 bits, 32 bits, or even 64 bits.
- **Real type:** Floating-point values.
- **Character type:** Sometimes treated as integers encoded in 8 or 16 bits.

Operators

Operations on numbers

Arithmetic Operations

ALGO	Langage C
+ : addition	+
- : substraction	-
* : multiplication	*
/ : decimal division	/
div : Integer division	/
mod : (modulo) remainder of division	%
- (unary minus) : negates the value of the operand	- (unary)

Operators

Operations on numbers

Comparison Operations

ALGO	Langage C
< : Checks if one number is less than another	<
> : Checks if one number is greater than another	>
<= : Checks if one number is less than or equal to another	<=
>= : Checks if one number is greater than or equal to another	>=
= : Checks if two numbers are equal	==
<> : Checks if two numbers are different	!=

Note: The results of comparison operations are **boolean**.

Operators

Logical operators: are used to perform logical operations, typically on boolean values. The common logical operators are:

ALGO	Langage C
NOT : logical NOT	!
AND : logical AND	&&
OR : Logical OR	

Comments

- Comments can be added to an algorithm to clarify or explain the functioning of certain parts of the algorithm.
- Comments are ignored during execution.

Syntax:

Single-line comment:

```
// comment here
```

Multi-line comment:

```
/*comment here  
comment here  
comment here  
*/
```

Assignment

Assignment is the operation of giving a value to a variable.

Syntax:

- ALGO : **<variable> ← <expression> ;**
- C : **<variable> = <expression> ;**

It consists of two parts:

- Left side of the assignment: **variable**
- Right side of the assignment: **evaluable expression**:
 - constant
 - value of a variable
 - result of an arithmetic or logical expression

Note

The left and right sides of the assignment must be of the **same** or **compatible type**.

Assignment

Example 1:

```
Algorithm affectation_1 ;
    Var      x, y, z : integer ;
            a, b, c : boolean ;

Begin
    x ← 5 ; // put the value 5 in x;
            // content of x is 5
    y ← x ; /* put (or copy) the content of
            x into y;
            x and y contain the same value
            */
    x ← x+1 ; /* add 1 to the content of x and
            put the result into x; the
            content of x is 6
            */

    z ← x+2*y ; /* calculate the expression and then
            put the result into z
            */
    a ← true ; // put the value true into a
    b ← false ; // put the value false into a
    c ← a OR b ; /* a or b is true according to the
            truth table, so the content of c will be
            true*/

End.
```

Assignement

Exemple 1 en C :

```
int main(){
    int x, y, z;
    x=5 ;      /* mettre dans x la valeur 5 ;
                le contenu de x est 5 */
    y=x ;      /* mettre (ou copier) le contenu de x dans y ;
                x et y contiennent la même valeur, y contient 5 */
    x= x+1 ; /* rajouter 1 au contenu de x et mettre le
                résultat dans x ;
                le contenu de x est 6 */
    z=x+2*y ; /* calculer l'expression puis mettre le résultat
                dans z, z contient 16*/
    return 0;
}
```

Assignement

Exemple 2:

Algorithm affectation_2 ;

```
Var      x : real ;  
         a : integer ;  
         b : boolean;
```

begin

```
  x ← 1 ; // put the value 1.0 into x;  
a ← x ; // incorrect assignment: a cannot receive a real number  
  a ← 2 ; // put the value 2 into a;  
  x ← a ; // correct assignment: the real type includes the  
           // integer type  
a ← 1.0 ; // incorrect assignment: a cannot receive  
           // a real number  
  b ← x>a; // put the value false into b
```

end.

Write statement

The **Write** statement allows displaying on the screen:

- A textual message (string of characters)
- A variable
- Multiple variables
- Or a textual message with variables

Syntax :

ALGO :

- `Write(<variable>) ;`
- `Write(<list of variables>) ; // separated by commas`
- `Write(<string_of_characters>) ;`

Exemple :

```
age ← 19; a ← 2; b ← 5; c ← a + b; // assuming the variables
                                   // are declared

Write("my age is:") ;
Write(age) ;
Write("my age is:", age) ; // displays: my age is 19
Write(a, "+", b, "=", c) ; // displays: 2+5=7
```

The Write instruction in C: printf

printf is a function that allows displaying a string of characters on the screen, in which values of variables can be inserted.

- The 1st parameter of **printf** is a string of characters, and it is required. The other parameters (optional) are the variables we want to display.
- For each variable, there must be a formatting character preceded by %.
- **printf** requires adding the line **#include<stdio.h>** at the beginning of the program.

```
int a=5; float b=6.2; char c='A';
```

```
printf ("An integer: %d, a real number: %f, a character: %c.", a, b, c);
```

The value of a will replace the first %d

The value of b will replace the %f

The value of c replace the %c

Display:

An integer: 5, a real number: 6.200, a character: A

The Write instruction in C: printf

Example :

```
#include<stdio.h>
int main (){
    int age, a, b, c;
    age = 19; a = 2; b = 5;
    c = a + b;
    printf ("mon age est:");
    printf ("%d\n", age);
    printf ("mon age est %d\n", age); // displays: mon age est 19
    printf ("%d+%d=%d", a, b, c); //displays: 2+5=7
}
```

The Write instruction in C: printf

Formatting variables with printf

Variable types		Formatting
Algorithm	C language	
Integer	int	%d
	long	%ld
real	float	%f
	double	%lf
character	char	%c

Read instruction

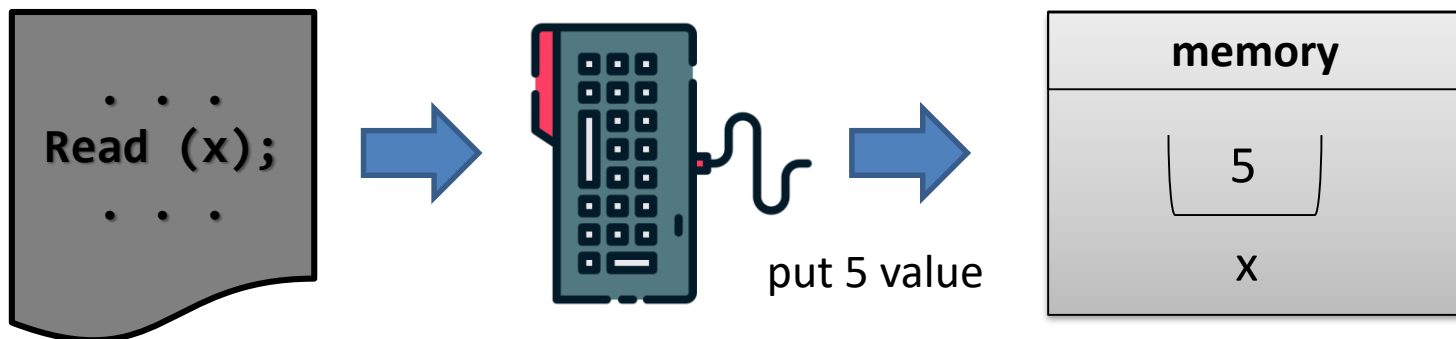
The read instruction commands the machine to read or take a data (value) entered **via the keyboard** and store it in a variable in memory.

Thus, read allows the user of a program to provide data typed on the keyboard and make it available to the program in a variable.

We can consider read as an assignment of a value entered via the keyboard to a variable stored in memory.

Syntax:

- ALGO : **Read(<variable>) ; or Read (<list of variables>) ;**



Read instruction - example

```
Algorithm test_read_write;  
var  
    a,b :integer;  
begin  
    Write ("donner deux valeurs entières:");  
    Read(a,b);  
    c ← a+b;  
    write(a, "+",b, "=", c) ;  
End.
```

Read instruction in C: scanf

Scanf is a function that allows retrieving data entered from the keyboard and storing it in a variable according to a format.

- The first parameter of **scanf** is a string of characters that contains the format of the variable to be read.
- The other parameters are the variables that we want to read.
- If the variable is of type integer, real number, or character, it must be preceded by &
- For each variable, there must be a corresponding formatting character preceded by % in the first parameter.
- **scanf** requires adding the line **#include<stdio.h>** at the beginning of the program.

scanf - example

```
Algorithm test_read_write;  
var  
    a,b :integer;  
begin  
    Write ("donner deux valeurs entières:");  
    Read(a,b);  
    c ← a+b;  
    write(a, "+",b, "=", c) ;  
End.
```

```
#include<stdio.h>  
int main () {  
    int a,b,c;  
    printf ("Donner deux valeurs entières:\n");  
    scanf ("%d", &a);  
    scanf ("%d", &b);  
    c=a+b;  
    printf ("%d+%d=%d", a,b,s);  
}
```